



UNIVERZITET U NIŠU
ELEKTRONSKI FAKULTET



Horizontalno particionisanje podataka (Sharding) u savremenim DBMS sistemima

Seminarski rad
Studijski program: Elektrotehnika i računarstvo
Modul: Bezbednost računarskih sistema
Predmet: Sistemi za upravljanje bazama podataka

Student:

Katarina Adamović, br. ind. 2173

Mentor:

Prof. dr Aleksandar Stanimirović

Niš, maj 2026.

SADRŽAJ:

1. Uvod	3
2. Skaliranje baza podataka — osnovni pojmovi	3
2.1. Vertikalno i horizontalno skaliranje	4
2.2. Replikacija i particionisanje	4
2.3. Horizontalno i vertikalno particionisanje	5
2.4. Razlika između particionisanja i shardinga	5
3. Arhitektura shardovanih sistema	6
3.1. Ključ raspodele (shard key)	6
3.2. Sloj rutiranja i katalog metapodataka	6
3.3. Kolokacija podataka	7
4. Strategije horizontalne raspodele podataka	7
4.1. Raspodela po opsegu (range sharding)	8
4.2. Raspodela heširanjem (hash sharding)	8
4.3. Konzistentno heširanje i virtuelni čvorovi	8
4.4. Raspodela preko imenika (directory-based sharding)	9
4.5. Geografska raspodela i kompozitne strategije	9
5. Izazovi horizontalnog pozicioniranja podataka	10
5.1. Distribuirani upiti	10
5.2. Distribuirane transakcije i konzistentnost	10
5.3. Rebalansiranje i promena šeme raspodele (resharding)	11
5.4. Operativna složenost	12
6. Sharding u savremenim DBMS sistemima	12
6.1. MongoDB	12
6.2. Apache Cassandra	12
6.3. Vitess	13
6.4. Citus (PostgreSQL)	13
6.5. NewSQL sistemi: Google Spanner i CockroachDB	13
7. Praktična implementacija u sistemu PostgreSQL	14
7.1. Deklarativno particionisanje unutar jedne instance	14
7.2. Sharding preko više servera pomoću postgres_fdw	21
7.3. Citus: distribuirani PostgreSQL klaster	27
7.4. Rad u alatu pgAdmin — rezime	34
8. Prednosti, nedostaci i preporuke za primenu	34
8.1. Prednosti	34
8.2. Nedostaci	34
8.3. Kada (ne) uvoditi sharding	35
9. Zaključak	35
Literatura	37

1. Uvod

Količina podataka koju savremeni informacijski sistemi generišu i obrađuju raste eksponencijalno. Društvene mreže, elektronska trgovina, IoT senzori, industrijski sistemi i finansijske platforme svakodnevno proizvode terabajte novih zapisa. Tradicionalni pristup, u kojem se čitava baza podataka nalazi na jednom serveru, u ovakvim uslovima brzo dostiže svoje granice — bilo da je reč o kapacitetu skladišta, propusnom opsegu diska, količini radne memorije ili broju upita koje sistem može da obradi u jedinici vremena.

Jedan od najvažnijih odgovora na ovaj problem jeste horizontalno pozicioniranje podataka, u literaturi poznato pod engleskim terminom sharding (šarding). Reč je o tehnici kojom se jedna logička baza podataka deli na više manjih, nezavisnih delova — šardova (engl. shards) — koji se raspoređuju na različite fizičke servere. Svaki šard sadrži podskup redova iste logičke tabele, a sistem kao celina i dalje predstavlja jedinstvenu bazu podataka iz perspektive aplikacije.

Cilj ovog seminarskog rada je da sistematično prikaže koncept horizontalnog pozicioniranja podataka: od osnovnih pojmova skaliranja baza podataka, preko arhitekture shardovanih sistema i strategija raspodele podataka, do izazova koje sharding donosi u domenu distribuiranih upita i transakcija. Poseban akcenat stavljen je na način na koji savremeni sistemi za upravljanje bazama podataka (DBMS) implementiraju sharding, uz praktične primere realizovane u sistemu PostgreSQL korišćenjem alata pgAdmin.

Rad je organizovan na sledeći način: u drugom poglavlju definisani su osnovni pojmovi skaliranja i particionisanja; treće poglavlje opisuje arhitekturu shardovanih sistema i problem izbora ključa raspodele; četvrto poglavlje analizira strategije shardinga; peto poglavlje razmatra izazove distribuiranih upita, transakcija i rebalansiranja; šesto poglavlje daje pregled implementacija u savremenim DBMS sistemima; sedmo poglavlje sadrži praktičnu implementaciju u PostgreSQL okruženju; osmo poglavlje sumira prednosti i nedostatke, dok deveto poglavlje donosi zaključna razmatranja.

2. Skaliranje baza podataka — osnovni pojmovi

2.1. Vertikalno i horizontalno skaliranje

Kada sistem za upravljanje bazom podataka dostigne granice svojih resursa, postoje dva fundamentalno različita pravca proširenja kapaciteta.

Vertikalno skaliranje (engl. scale-up) podrazumeva jačanje postojećeg servera: dodavanje procesorskih jezgara, radne memorije, brzih NVMe diskova ili prelazak na moćniju hardversku platformu. Ovaj pristup je jednostavan jer ne zahteva izmene u arhitekturi aplikacije — baza ostaje jedna, transakcije ostaju lokalne, a administracija se ne menja. Međutim, vertikalno skaliranje ima dve ključne mane: cena hardvera raste nelinearno (server dvostruko veće snage košta znatno više nego dvostruko), a fizičke granice jednog računara su konačne. Pored toga, jedan server predstavlja jedinstvenu tačku otkaza (engl. single point of failure).

Horizontalno skaliranje (engl. scale-out) podrazumeva dodavanje novih, tipično jeftinijih servera (engl. commodity hardware) i raspodelu opterećenja među njima. Kapacitet sistema tada raste približno linearno sa brojem čvorova, a otkaz jednog čvora ne mora značiti nedostupnost celog sistema. Cena ovog pristupa je značajno povećanje složenosti: podaci se moraju raspodeliti, upiti usmeriti na prave čvorove, a konzistentnost održati preko mreže.

Kriterijum	Vertikalno skaliranje	Horizontalno skaliranje
Način proširenja	jači hardver na jednom serveru	dodavanje novih servera
Gornja granica	fizički limit jedne mašine	praktično neograničena
Cena rasta	nelinearna (skupi serveri)	približno linearna
Složenost aplikacije	niska	visoka (rutiranje, distribucija)
Otpornost na otkaze	jedinstvena tačka otkaza	otkaz čvora je izolovan
Transakcije	lokalne, jednostavne	distribuirane, složene

Tabela 1. Poređenje vertikalnog i horizontalnog skaliranja

2.2. Replikacija i particionisanje

Horizontalno skaliranje počiva na dve komplementarne tehnike: replikaciji i particionisanju.

Replikacija podrazumeva čuvanje istih podataka na više čvorova. Njena primarna svrha je visoka dostupnost (podaci preživljavaju otkaz čvora) i skaliranje operacija čitanja (upiti se mogu opslužiti sa bilo koje replike). Replikacija, međutim, ne rešava problem kapaciteta — svaki čvor i dalje mora da smesti kompletan skup podataka — niti skalira operacije pisanja, jer se svaki upis mora propagirati na sve replike.

Particionisanje podrazumeva deljenje skupa podataka na disjunktne delove, tako da svaki čvor čuva samo deo podataka. Time se skaliraju i kapacitet skladišta i propusnost pisanja, jer se upisi raspodeljuju po čvorovima. U praksi se replikacija i particionisanje gotovo uvek kombinuju: svaki šard se dodatno replicira, čime se postiže i skalabilnost i otpornost na otkaze [1].

2.3. Horizontalno i vertikalno particionisanje

Samo particionisanje može biti izvedeno na dva načina, u zavisnosti od toga po kojoj se dimenziji tabela deli.

Vertikalno particionisanje deli tabelu po kolonama: različite grupe kolona smeštaju se u različite tabele ili na različite servere, pri čemu sve particije dele isti primarni ključ. Ovaj pristup je koristan kada se pojedine kolone (npr. veliki BLOB sadržaji) retko koriste, pa se njihovim izdvajanjem smanjuje obim podataka koji se čita u tipičnim upitima. Srodan koncept na nivou celog sistema je funkcionalna dekompozicija, gde se različite tabele (npr. korisnici, proizvodi, porudžbine) smeštaju u zasebne baze — što je čest obrazac u mikroservisnim arhitekturama.

Horizontalno particionisanje deli tabelu po redovima: svaka particija ima identičnu šemu (iste kolone), ali sadrži različit podskup redova, određen vrednošću izabranog atributa. Upravo je horizontalno particionisanje, primenjeno preko više nezavisnih servera, ono što nazivamo shardingom.

2.4. Razlika između particionisanja i shardinga

U literaturi i dokumentaciji različitih sistema termini particionisanje (engl. partitioning) i sharding često se koriste kao sinonimi, ali postoji ustaljena distinkcija [2]:

- **Particionisanje** najčešće označava deljenje tabele na više fizičkih delova unutar jedne instance DBMS-a, na istom serveru. Cilj je pre svega bolja organizacija podataka, efikasnije održavanje (npr. brisanje starih particija) i ubrzanje upita eliminacijom nepotrebnih particija (engl. partition pruning).
- **Sharding** označava raspodelu particija preko više nezavisnih servera (čvorova), od kojih svaki poseduje sopstvene resurse — procesor, memoriju i disk. Cilj je skaliranje kapaciteta i propusnosti izvan granica jedne mašine.

Drugim rečima, sharding je horizontalno particionisanje podignuto na nivo distribuiranog sistema. Ova razlika će biti posebno vidljiva u praktičnom delu rada: PostgreSQL nudi deklarativno particionisanje

unutar jedne instance, a shardovanje preko više instanci ostvaruje se mehanizmima poput postgres_fdw ili ekstenzije Citus (poglavlje 7).

3. Arhitektura shardovanih sistema

3.1. Ključ raspodele (shard key)

Centralna odluka pri projektovanju shardovanog sistema jeste izbor ključa raspodele (engl. shard key ili partition key) — atributa (ili skupa atributa) na osnovu čije vrednosti se određuje kojem šardu red pripada. Funkcija raspodele preslikava vrednost ključa u identifikator šarda, a kvalitet tog preslikavanja direktno određuje performanse celog sistema.

Dobar ključ raspodele treba da ispuni nekoliko uslova:

- **Visoka kardinalnost** — ključ mora imati dovoljno različitih vrednosti da bi se podaci uopšte mogli podeliti na veliki broj šardova. Kolona pol sa dve moguće vrednosti ne može podržati više od dva šarda.
- **Ravnomerna raspodela** — vrednosti ključa treba da budu približno uniformno zastupljene, kako bi šardovi bili ujednačene veličine. Neravnomerna raspodela dovodi do iskrivljenja (engl. data skew) i pojave „vrućih tačaka“ (engl. hotspots) — šardova koji primaju nesrazmerno veliki deo opterećenja.
- **Uskladenost sa obrascima pristupa** — najčešći upiti treba da sadrže ključ raspodele u uslovu, kako bi sistem mogao da ih usmeri na tačno jedan šard. Upit koji ne sadrži ključ mora se izvršiti na svim šardovima (engl. scatter-gather), što poništava veliki deo dobitka.
- **Stabilnost** — vrednost ključa se po pravilu ne sme menjati, jer bi promena zahtevala fizičko premeštanje reda sa jednog šarda na drugi.

Klasičan primer lošeg izbora je vremenska oznaka kreiranja zapisa u sistemu sa opsegovnom raspodelom: svi novi upisi završavaju na istom (poslednjem) šardu, koji postaje usko grlo, dok ostali šardovi miruju. Sličan problem je i tzv. „problem slavne ličnosti“ (engl. celebrity problem) u društvenim mrežama: ako je ključ raspodele identifikator korisnika, nalog sa milionima pratilaca generiše ekstremno opterećenje na jednom šardu, bez obzira na kvalitet funkcije raspodele [1].

3.2. Sloj rutiranja i katalog metapodataka

Pošto su podaci fizički raspoređeni na više servera, sistem mora da odgovori na pitanje: kako upit stiže do pravog šarda? U praksi se sreću tri arhitekturna obrasca [3]:

- **Rutiranje u aplikaciji** — aplikacija (ili njen sloj za pristup podacima) sama izračunava ciljni šard i otvara konekciju direktno ka njemu. Pristup je jednostavan i bez dodatnih komponenti, ali logika raspodele „curi“ u aplikativni kod i mora se ažurirati pri svakoj promeni topologije.
- **Posrednički (proxy) sloj** — između aplikacije i baza stoji specijalizovana komponenta (npr. mongos u MongoDB-u, VTGate u Vitess-u) koja prima upite, konsultuje katalog metapodataka i prosleđuje ih odgovarajućim šardovima, a zatim objedinjuje rezultate. Aplikacija vidi sistem kao jednu logičku bazu.
- **Koordinatorski čvor** — jedan od čvorova DBMS-a ima ulogu koordinatora: prima upite, planira njihovo distribuirano izvršavanje i delegira fragmente radnicima (engl. worker nodes). Ovaj model koristi Citus ekstenzija za PostgreSQL.

U svim varijantama postoji katalog metapodataka — mapa koja povezuje opsege ili heš-vrednosti ključa sa konkretnim šardovima i njihovim mrežnim adresama. Katalog mora biti visoko dostupan i konzistentan, jer njegova nedostupnost znači nedostupnost celog sistema; zato se često čuva u zasebnom, repliciranom skladištu (npr. config serveri u MongoDB-u, etcd/ZooKeeper u drugim sistemima).

3.3. Kolokacija podataka

Važan arhitekturni koncept je **kolokacija** (engl. co-location): tabele koje se često spajaju treba shardovati po istom ključu, tako da povezani redovi završe na istom čvoru. Na primer, u višekorisničkom (multi-tenant) SaaS sistemu prirodno je sve tabele shardovati po identifikatoru klijenta (tenant_id) — tada se svi podaci jednog klijenta nalaze na jednom šardu, pa se spajanja i transakcije u okviru klijenta izvršavaju lokalno, bez mrežne komunikacije. Male, retko menjane šifarničke tabele (npr. lista država) često se u celosti repliciraju na sve čvorove kao referentne tabele, čime se omogućava lokalno spajanje sa bilo kojim šardom [4].

4. Strategije horizontalne raspodele podataka

Strategija shardinga definiše funkciju koja vrednost ključa raspodele preslikava u konkretan šard. Izbor strategije predstavlja kompromis između ravnomernosti raspodele, efikasnosti opsegovnih upita i

složenosti rebalansiranja.

4.1. Raspodela po opsegu (range sharding)

Kod raspodele po opsegu, domen ključa se deli na kontinualne intervale, a svaki interval se dodeljuje jednom šardu. Na primer, korisnici sa prezimenima A–H idu na šard 1, I–P na šard 2, R–Š na šard 3; ili porudžbine iz 2024. godine na jedan, a iz 2025. na drugi šard.

Glavna prednost ovog pristupa je efikasnost **opsegovnih upita**: upit koji traži sve porudžbine iz jednog kvartala pogađa tačno jedan ili nekoliko susednih šardova. Katalog metapodataka je kompaktan (spisak granica intervala), a dodavanje novih opsega je jednostavno. Glavna mana je sklonost ka neravnomernoj raspodeli i vrućim tačkama: ako je ključ monotono rastući (vreme, sekvencijalni ID), sva pisanja pogađaju poslednji šard. Sistemi poput Google Spanner-a i CockroachDB-a koriste dinamičku varijantu ovog pristupa, u kojoj se opsezi automatski dele i premeštaju kada postanu preveliki ili preopterećeni [5].

4.2. Raspodela heširanjem (hash sharding)

Kod heš raspodele, nad vrednošću ključa se izračunava heš funkcija, a rezultat se preslikava u šard — najjednostavnije po formuli $\text{shard} = \text{hash}(k) \bmod N$, gde je N broj šardova. Kvalitetna heš funkcija raspršuje i međusobno bliske vrednosti ključa uniformno po celom opsegu, čime se postiže ravnomerna raspodela podataka i opterećenja, bez vrućih tačaka čak i za monotone ključeve.

Cena ove uniformnosti je gubitak lokalnosti: susedne vrednosti ključa završavaju na različitim šardovima, pa se opsegovni upiti moraju izvršiti na svim šardovima. Dodatni problem klasične modulo formule je rebalansiranje: promena broja šardova sa N na $N+1$ menja rezultat za gotovo sve ključeve, što bi zahtevalo premeštanje skoro svih podataka.

4.3. Konzistentno heširanje i virtuelni čvorovi

Problem masovnog premeštanja rešava konzistentno heširanje (engl. consistent hashing) [6]. Prostor heš vrednosti se posmatra kao prsten; i čvorovi i ključevi se heširaju na prsten, a ključ pripada prvom čvoru na koji se naiđe kretanjem u smeru kazaljke. Dodavanje ili uklanjanje čvora tada pomera samo ključeve između tog čvora i njegovog suseda — u proseku $1/N$ ukupnih podataka — umesto gotovo svih.

Da bi raspodela bila ravnomernija, svaki fizički čvor se na prsten postavlja u više tačaka, kao skup virtuelnih čvorova (engl. virtual nodes, vnodes). Ovaj mehanizam koriste Amazon Dynamo i Apache Cassandra [7]. Srodna, u praksi vrlo česta tehnika je raspodela u fiksni broj logičkih particija: sistem se unapred podeli na mnogo više particija nego što ima čvorova (npr. 32 čvora, 4096 particija), a rebalansiranje se svodi na premeštanje celih particija između čvorova, bez ponovnog heširanja ključeva.

4.4. Raspodela preko imenika (directory-based sharding)

Kod imeničke raspodele, preslikavanje ključa (ili grupe ključeva) u šard čuva se eksplicitno, u posebnoj tabeli pretraživanja (engl. lookup table). Ovaj pristup pruža maksimalnu fleksibilnost: pojedinačni entitet se može premestiti na drugi šard prostom izmenom zapisa u imeniku, što je korisno npr. za izolovanje izuzetno velikog klijenta na namenski server. Mane su dodatni mrežni skok pri svakom pristupu (ili potreba za keširanjem imenika) i činjenica da imenik postaje kritična komponenta koja se mora replicirati i održavati konzistentnom.

4.5. Geografska raspodela i kompozitne strategije

U globalno distribuiranim sistemima ključ raspodele često uključuje geografsku komponentu: podaci evropskih korisnika smeštaju se u evropski data-centar, američkih u američki. Time se smanjuje latencija pristupa i olakšava usklađenost sa regulativom o lokalizaciji podataka (npr. GDPR). U praksi se strategije često kombinuju u kompozitne ključeve: na primer, Cassandra particioniše po heš vrednosti particionog ključa, a unutar particije sortira redove po klaster-kolonama, čime kombinuje uniformnost heširanja sa efikasnim opsegovnim upitima unutar particije [7].

Strategija	Prednosti	Nedostaci	Tipični predstavnici
Po opsegu	efikasni opsegovni upiti; jednostavan katalog	vruće tačke kod monotonihi ključeva; skew	Spanner, CockroachDB, HBase
Heširanjem	ravnomerna raspodela; nema vrućih tačaka	opsegovni upiti pogađaju sve šardove	Citus, MongoDB (hashed), YugabyteDB
Konzistentno heširanje	minimalno premeštanje pri promeni broja čvorova	složenija implementacija	Cassandra, DynamoDB, Riak
Preko imenika	potpuna fleksibilnost	dodatni skok; kritična	prilagođena rešenja,

	premeštanja	lookup komponenta	Vitess (delom)
Geografska	niska latencija; regulativa	neravnomerno opterećenje regiona	Spanner, CockroachDB (multi-region)

Tabela 2. Uporedni pregled strategija shardinga

5. Izazovi horizontalnog pozicioniranja podataka

Sharding rešava problem kapaciteta i propusnosti, ali po cenu značajnog usložnjavanja gotovo svih aspekata rada sa bazom. Ova sekcija razmatra četiri ključna izazova: distribuirane upite, distribuirane transakcije, rebalansiranje podataka i operativnu složenost.

5.1. Distribuirani upiti

Upit koji u uslovu sadrži ključ raspodele sistem usmerava na tačno jedan šard (engl. targeted query) i njegova cena je praktično jednaka ceni upita u neshardovanoj bazi. Problem nastaje sa upitima koji ključ ne sadrže: oni se moraju poslati svim šardovima, a parcijalni rezultati objediniti — obrazac poznat kao scatter-gather. Latencija takvog upita određena je najsporijim šardom, a njegova cena raste sa brojem čvorova, pa sistemi sa velikim udelom scatter-gather upita loše skaliraju.

Posebno su problematična spajanja preko šardova (engl. cross-shard join): ako se dve tabele spajaju po atributu koji nije ključ raspodele, sistem mora da premešta međurezultate između čvorova (engl. data shuffling), što je skupo i po mrežnom saobraćaju i po memoriji. U praksi se ovaj problem ublažava na tri načina:

- **Kolokacijom** — tabele koje se često spajaju shardaju se po istom ključu, pa se spajanje izvršava lokalno na svakom čvoru (videti odeljak 3.3);
- **Referentnim tabelama** — male tabele se repliciraju na sve čvorove, čime svako spajanje sa njima postaje lokalno;
- **Denormalizacijom** — podaci se namerno dupliraju u okviru istog šarda kako bi se spajanje u potpunosti izbeglo, po cenu složenijeg održavanja konzistentnosti kopija.

Agregatne funkcije zahtevaju dekompoziciju na lokalni i globalni korak: SUM i COUNT se računaju lokalno pa sabiraju, AVG se izvodi iz distribuiranih suma i brojeva redova, dok su operacije poput globalnog sortiranja sa OFFSET paginacijom ili COUNT(DISTINCT) znatno skuplje jer zahtevaju prenos većih međurezultata na koordinator.

5.2. Distribuirane transakcije i konzistentnost

ACID transakcija koja menja podatke na jednom šardu izvršava se standardnim lokalnim mehanizmima. Transakcija koja obuhvata više šardova, međutim, zahteva **protokol distribuiranog potvrđivanja**. Klasično rešenje je **dvofazno potvrđivanje** (engl. two-phase commit, 2PC): u prvoj fazi koordinator traži od svih učesnika da pripreme transakciju i garantuju da je mogu potvrditi; u drugoj fazi, ako su svi odgovorili potvrdno, šalje konačnu odluku o potvrđivanju [8]. Protokol garantuje atomičnost, ali unosi dodatne mrežne runde, drži brave tokom čitavog trajanja protokola i blokira učesnike ako koordinator otkaže u kritičnom trenutku — zbog čega se transakcije preko šardova u praksi svode na najmanju moguću meru.

Širi teorijski okvir daje **CAP teorema** [9]: u prisustvu mrežne particije (P), distribuirani sistem mora da bira između konzistentnosti (C) i dostupnosti (A). Sistemi poput Cassandre biraju dostupnost uz podesivu, na kraju usaglašenu konzistentnost (engl. eventual consistency), dok NewSQL sistemi poput Spanner-a i CockroachDB-a biraju strogu konzistentnost, koristeći konsenzus protokole (Paxos, Raft) za replikaciju i sofisticirane mehanizme (kod Spanner-a globalno sinhronizovane časovnike TrueTime) za serijabilnost transakcija [5]. Praktična posledica po projektovanje shardovanih sistema je jasna: šema i ključ raspodele treba da budu izabrani tako da ogromna većina transakcija ostane u granicama jednog šarda.

Dodatni, često potcenjen problem je **globalna jedinstvenost identifikatora**: klasične auto-increment sekvence ne funkcionišu preko više nezavisnih čvorova. Rešenja uključuju UUID vrednosti, kompozitne ključeve (id šarda + lokalna sekvenca), centralizovane servise za dodelu identifikatora, ili strukturirane 64-bitne identifikatore sa vremenskom komponentom (poput Snowflake ID formata).

5.3. Rebalansiranje i promena šeme raspodele (resharding)

Raspodela podataka nije statična: podaci rastu, obrasci pristupa se menjaju, čvorovi se dodaju i uklanjaju. Rebalansiranje je proces premeštanja podataka između čvorova radi ujednačavanja opterećenja, a resharding je promena same šeme raspodele (npr. broja šardova ili ključa).

Ključni zahtevi za rebalansiranje su: (1) da se premešta minimalna količina podataka, (2) da sistem tokom premeštanja ostane dostupan (engl. online rebalancing) i (3) da se opterećenje samog premeštanja kontroliše kako ne bi ugušilo produkcionu saobraćaj. Tehnike koje to omogućavaju već su pomenute: konzistentno heširanje, fiksni broj logičkih particija znatno veći od broja čvorova, i dinamičko deljenje opsega. Tipičan online postupak podrazumeva kopiranje particije na novi čvor uz kontinuiranu primenu pristiglih izmena (praćenjem write-ahead loga), kratku sinhronizaciju i atomsko

preusmeravanje kataloga metapodataka na novu lokaciju [1]. Najskuplja operacija je promena ključa raspodele, koja u opštem slučaju zahteva potpunu preraspodelu svih podataka — zbog čega se izboru ključa posvećuje tolika pažnja na početku projektovanja.

5.4. Operativna složenost

Shardovan sistem je distribuiran sistem, sa svim operativnim posledicama: rezervne kopije se moraju praviti i vremenski usaglašavati preko više čvorova; izmene šeme (migracije) moraju se sprovoditi koordinisano na svim šardovima; nadzor mora pratiti ne samo zdravlje pojedinačnih čvorova već i ravnomernost raspodele; testiranje i razvojna okruženja postaju složenija. Zbog svega navedenog, opšteprihvaćena preporuka glasi: **sharding uvoditi tek kada su iscrpljene jednostavnije opcije** — optimizacija upita i indeksa, vertikalno skaliranje, keširanje i replike za čitanje [2].

6. Sharding u savremenim DBMS sistemima

Savremeni DBMS sistemi implementiraju horizontalno particionisanje na različite načine — od potpuno automatskog, transparentnog shardinga u NoSQL i NewSQL sistemima, do ekstenzija i posredničkih slojeva koji sharding dodaju klasičnim relacionim sistemima. U nastavku je dat pregled reprezentativnih rešenja.

6.1. MongoDB

MongoDB, najrasprostranjenija dokument baza, ima sharding ugrađen u jezgro arhitekture. Shardovan klaster čine tri vrste komponenti: shard serveri (replika-setovi koji čuvaju podatke), config serveri (replirani katalog metapodataka) i mongos ruteri (proxy procesi na koje se aplikacija povezuje) [10]. Kolekcija se shardaje izborom shard ključa, uz podršku i za opsegovnu i za heš raspodelu. Podaci se interno dele na chunk-ove (podrazumevano do 128 MB), a pozadinski balancer proces automatski premešta chunk-ove između šardova radi ujednačavanja. Od verzije 5.0 podržano je i naknadno menjanje shard ključa (resharding) uz rad sistema.

6.2. Apache Cassandra

Cassandra je predstavnik decentralizovane, peer-to-peer arhitekture izvedene iz Amazon Dynamo dizajna [7]. Ne postoji koordinator ni posebni meta-čvorovi: svi čvorovi su ravnopravni, a podaci se

raspodeljuju konzistentnim heširanjem particionog ključa na token-prsten, uz virtuelne čvorove radi ravnomernosti. Svaka particija se replicira na konfigurisan broj čvorova (faktor replikacije), a konzistentnost je podesiva po upitu (ONE, QUORUM, ALL). Cassandra žrtvuje bogatstvo upitnog modela (nema spajanja, upiti su vezani za particioni ključ) u korist linearne skalabilnosti pisanja i visoke dostupnosti.

6.3. Vitess

Vitess je sloj za horizontalno skaliranje MySQL-a, razvijen u YouTube-u i danas CNCF projekat. Aplikacija se povezuje na VTGate proxy, koji na osnovu šeme raspodele (VSchema) rutira upite ka VTablet procesima ispred pojedinačnih MySQL instanci. Vitess podržava resharding uživo, upravljanje topologijom preko etcd/ZooKeeper skladišta i koristi se u sistemima ekstremnih razmera (YouTube, Slack, GitHub) [11].

6.4. Citus (PostgreSQL)

Citus je ekstenzija otvorenog koda koja PostgreSQL pretvara u distribuiranu bazu. Klaster čine koordinator i radni čvorovi (workers); tabela se distribuira pozivom funkcije `create_distributed_table`, kojom se bira kolona raspodele, a podaci se heširanjem dele na konfigurisan broj šardova (podrazumevano 32), fizički realizovanih kao obične PostgreSQL tabele na radnim čvorovima [4]. Citus podržava kolokaciju tabela, referentne tabele, distribuirane transakcije preko 2PC protokola i paralelno izvršavanje analitičkih upita. Microsoft (koji je preuzeo kompaniju Citus Data 2019. godine) nudi Citus kao upravljaju uslugu u Azure oblaku. Citus je korišćen u praktičnom delu ovog rada (odeljak 7.3).

6.5. NewSQL sistemi: Google Spanner i CockroachDB

NewSQL sistemi nastoje da pomire horizontalnu skalabilnost NoSQL sveta sa relacionim modelom i ACID transakcijama. Google Spanner deli podatke na opsege (splits) koji se automatski dele i premeštaju, replicira ih Paxos protokolom preko data-centara i koristi TrueTime API — globalno sinhronizovane časovnike sa ograničenom neizvesnošću — za spolja serijabilne (externally consistent) transakcije na globalnom nivou [5]. CockroachDB, inspirisan Spanner-om, deli prostor ključeva na opsege od po ~512 MB, replicira ih Raft protokolom i automatski rebalansira, pružajući PostgreSQL-kompatibilan SQL interfejs bez potrebe da korisnik ručno upravlja šardovima. U ovim sistemima sharding je u potpunosti transparentan — korisnik radi sa logički jedinstvenom SQL bazom.

Sistem	Model	Strategija raspodele	Rutiranje	Transakcije preko šardova
MongoDB	dokument	opseg ili heš (chunks)	mongos + config serveri	da (od v4.2, 2PC)
Cassandra	wide-column	konzistentno heširanje	bilo koji čvor (peer-to-peer)	ne (eventual consistency)
Vitess/MySQL	relacioni	heš/opseg (VSchema)	VTGate proxy	ograničeno (2PC, opciono)
Citus/PostgreSQL	relacioni	heš; referentne tabele	koordinator	da (2PC)
Spanner	relacioni (NewSQL)	dinamički opsezi	transparentno	da (Paxos + TrueTime)
CockroachDB	relacioni (NewSQL)	dinamički opsezi	transparentno (bilo koji čvor)	da (Raft, serializable)

Tabela 3. Uporedni pregled shardinga u savremenim DBMS sistemima

7. Praktična implementacija u sistemu PostgreSQL

PostgreSQL ne poseduje ugrađen, automatski sharding preko više servera, ali nudi sve gradivne elemente za njegovu realizaciju: deklarativno particionisanje tabela, mehanizam stranih tabela (`postgres_fdw`) i ekosistem ekstenzija među kojima je najznačajniji Citus. U ovom poglavlju prikazana su tri nivoa praktične realizacije horizontalne raspodele, uz uputstva za rad u alatu pgAdmin. Kao radni primer koristi se pojednostavljena šema sistema porudžbina, sa tabelama kupaca i porudžbina.

7.1. Deklarativno particionisanje unutar jedne instance

Prvi korak ka horizontalnoj raspodeli je particionisanje tabele unutar jedne PostgreSQL instance. Od verzije 10 PostgreSQL podržava deklarativno particionisanje po opsegu (RANGE), listi (LIST) i hešu (HASH). Particionisana tabela je logički kontejner: podaci se fizički smeštaju u particije, a planer upita automatski eliminiše particije koje ne mogu sadržati tražene redove (engl. *partition pruning*).

Da bi se stekla jasnija slika o tome šta particionisanje unutar jedne instance zaista donosi u praksi, izvršeno je direktno poređenje između heš-particionisane tabele `porudzbina` i obične, neparticionisane tabele `porudzbina_obicna` iste veličine (po 1.000.000 redova), za tri tipa upita: ciljani upit po ključu particionisanja, upit bez uslova po ključu particionisanja i globalnu agregaciju nad celom tabelom. Tabela `porudzbina_obicna` je najpre napunjena sa 1.000.000 sintetičkih redova:

 seminarski_sharding/postgres@PostgreSQL 18

No limit

Query Query History AI Assistant

```
1 INSERT INTO porudzbina_obicna (kupac_id, datum, iznos)
2 SELECT (random() * 100000)::bigint + 1,
3        now() - (random() * interval '365 days'),
4        round((random() * 5000)::numeric, 2)
5 FROM generate_series(1, 1000000);
```

Data Output Messages Notifications

INSERT 0 1000000

Query returned successfully in 6 secs 679 msec.

Zatim je kreirana heš-particionisana tabela porudzbina sa četiri particije:

seminarski_sharding/postgres@PostgreSQL 18

Query Query History AI Assistant

```

1 CREATE TABLE porudzbina (
2     id          bigint GENERATED ALWAYS AS IDENTITY,
3     kupac_id    bigint NOT NULL,
4     datum       timestampz NOT NULL DEFAULT now(),
5     iznos       numeric(12,2) NOT NULL,
6     status      text NOT NULL DEFAULT 'NOVA',
7     PRIMARY KEY (id, kupac_id)
8 ) PARTITION BY HASH (kupac_id);
9
10 CREATE TABLE porudzbina_p0 PARTITION OF porudzbina
11     FOR VALUES WITH (MODULUS 4, REMAINDER 0);
12 CREATE TABLE porudzbina_p1 PARTITION OF porudzbina
13     FOR VALUES WITH (MODULUS 4, REMAINDER 1);
14 CREATE TABLE porudzbina_p2 PARTITION OF porudzbina
15     FOR VALUES WITH (MODULUS 4, REMAINDER 2);
16 CREATE TABLE porudzbina_p3 PARTITION OF porudzbina
17     FOR VALUES WITH (MODULUS 4, REMAINDER 3);

```

Data Output Messages Notifications

CREATE TABLE

Query returned successfully in 77 msec.

seminarski_sharding/postgres@PostgreSQL 18

Query Query History AI Assistant

```

1 SELECT relname FROM pg_class WHERE relname LIKE 'porudzbina%' ORDER BY relname;

```

Data Output Messages Notifications

Showing rows: 1

	relname name
1	porudzbina
2	porudzbina_id_seq
3	porudzbina_obicna
4	porudzbina_obicna_id_s...
5	porudzbina_obicna_pkey
6	porudzbina_p0
7	porudzbina_p0_pkey
8	porudzbina_p1
9	porudzbina_p1_pkey
10	porudzbina_p2
11	porudzbina_p2_pkey
12	porudzbina_p3
13	porudzbina_p3_pkey
14	porudzbina_pkey

Tabela porudzbina je zatim napunjena istim brojem redova (1.000.000):

seminarski_sharding/postgres@PostgreSQL 18

Query Query History AI Assistant

```

1 INSERT INTO porudzbina (kupac_id, datum, iznos)
2 SELECT (random() * 100000)::bigint + 1,
3        now() - (random() * interval '365 days'),
4        round((random() * 5000)::numeric, 2)
5 FROM generate_series(1, 1000000);

```

Data Output Messages Notifications

INSERT 0 1000000

Query returned successfully in 7 secs 560 msec.

Za ciljani upit — izračunavanje broja i zbira iznosa porudžbina jednog kupca (`kupac_id = 42`) — razlika je izražena. Nad običnom tabelom EXPLAIN ANALYZE pokazuje sekvencijalno skeniranje svih redova (Parallel Seq Scan on `porudzbina_obicna`, uz odbacivanje 333.331 reda koji ne zadovoljavaju uslov) i vreme izvršavanja od 104,518 ms:

seminarski_sharding/postgres@PostgreSQL 18

Query Query History AI Assistant

```

1 EXPLAIN ANALYZE
2 SELECT count(*), sum(iznos) FROM porudzbina_obicna WHERE kupac_id = 42;

```

Data Output Messages Notifications

Showing rows: 1 to 16

	QUERY PLAN
1	Finalize Aggregate (cost=14542.59..14542.60 rows=1 width=40) (actual time=97.504..104.483 rows=1.00 loops=1)
2	Buffers: shared hit=2757 read=5577
3	-> Gather (cost=14542.36..14542.57 rows=2 width=40) (actual time=97.359..104.473 rows=3.00 loops=1)
4	Workers Planned: 2
5	Workers Launched: 2
6	Buffers: shared hit=2757 read=5577
7	-> Partial Aggregate (cost=13542.36..13542.37 rows=1 width=40) (actual time=53.814..53.815 rows=1.00 loops=3)
8	Buffers: shared hit=2757 read=5577
9	-> Parallel Seq Scan on porudzbina_obicna (cost=0.00..13542.33 rows=5 width=6) (actual time=11.093..53.783 rows=2.67 loo...
10	Filter: (kupac_id = 42)
11	Rows Removed by Filter: 333331
12	Buffers: shared hit=2757 read=5577
13	Planning:
14	Buffers: shared hit=37 read=9
15	Planning Time: 1.555 ms
16	Execution Time: 104.518 ms

Particionisana tabela, zahvaljujući eliminaciji particija opisanoj iznad, pristupa samo particiji porudzbina_p2 i završava isti upit za 38,408 ms — oko 2,7 puta brže:

seminarski_sharding/postgres@PostgreSQL 18	
Query Query History AI Assistant	
<pre>1 EXPLAIN ANALYZE 2 SELECT count(*), sum(iznos) FROM porudzbina WHERE kupac_id = 42;</pre>	
Data Output Messages Notifications	
Showing rows: 1 to 16	
QUERY PLAN	
text	
1	Finalize Aggregate (cost=4920.72..4920.73 rows=1 width=40) (actual time=34.472..38.373 rows=1.00 loops=1)
2	Buffers: shared hit=2083
3	-> Gather (cost=4920.50..4920.71 rows=2 width=40) (actual time=20.022..38.364 rows=2.00 loops=1)
4	Workers Planned: 1
5	Workers Launched: 1
6	Buffers: shared hit=2083
7	-> Partial Aggregate (cost=3920.50..3920.51 rows=1 width=40) (actual time=9.908..9.908 rows=1.00 loops=2)
8	Buffers: shared hit=2083
9	-> Parallel Seq Scan on porudzbina_p2 porudzbina (cost=0.00..3920.46 rows=6 width=6) (actual time=0.035..9.895 rows=3.00 loo...
10	Filter: (kupac_id = 42)
11	Rows Removed by Filter: 124944
12	Buffers: shared hit=2083
13	Planning:
14	Buffers: shared hit=58
15	Planning Time: 1.606 ms
16	Execution Time: 38.408 ms

Slika se, međutim, menja kod upita koji ne sadrži ključ particionisanja. Za upit koji broji porudžbine sa statusom 'NOVA' (SELECT count(*) FROM ... WHERE status = 'NOVA'), obična tabela vraća rezultat za 144,505 ms jednim sekvencijalnim skeniranjem, dok particionisana tabela, čiji plan pokazuje čvor Parallel Append koji objedinjuje po jedan Parallel Seq Scan nad svakom od četiri particije (p0–p3), troši 236,885 ms — dakle sporije, iako obrađuje isti broj redova:

</

Isti obrazac ponavlja se i kod globalne agregacije (SELECT sum(iznos) FROM ...): obična tabela izvršava upit za 129,832 ms, a particionisana za 139,818 ms:

Query	Query History	AI Assistant
1	EXPLAIN ANALYZE	
2	SELECT sum(iznos) FROM porudzbina_obicna;	
Data Output	Messages	Notifications
Showing rows: 1 to 12		
QUERY PLAN		
text		
1	Finalize Aggregate (cost=14542.56..14542.57 rows=1 width=32) (actual time=125.031..129.784 rows=1.00 loops=1)	
2	Buffers: shared hit=3321 read=5013	
3	-> Gather (cost=14542.34..14542.55 rows=2 width=32) (actual time=124.896..129.774 rows=3.00 loops=1)	
4	Workers Planned: 2	
5	Workers Launched: 2	
6	Buffers: shared hit=3321 read=5013	
7	-> Partial Aggregate (cost=13542.34..13542.35 rows=1 width=32) (actual time=91.245..91.245 rows=1.00 loops=3)	
8	Buffers: shared hit=3321 read=5013	
9	-> Parallel Seq Scan on porudzbina_obicna (cost=0.00..12500.67 rows=416667 width=6) (actual time=0.456..28.171 rows=333333.33 loo...	
10	Buffers: shared hit=3321 read=5013	
11	Planning Time: 0.101 ms	
12	Execution Time: 129.832 ms	

Query	Query History	AI Assistant	Scratch f
1	EXPLAIN ANALYZE		
2	SELECT sum(iznos) FROM porudzbina;		
Data Output	Messages	Notifications	
Showing rows: 1 to 22			
QUERY PLAN			
text			
5	Workers Launched: 2		
6	Buffers: shared hit=8335		
7	-> Partial Aggregate (cost=17342.36..17342.37 rows=1 width=32) (actual time=104.151..104.152 rows=1.00 loops=3)		
8	Buffers: shared hit=8335		
9	-> Parallel Append (cost=0.00..16300.69 rows=416667 width=6) (actual time=0.012..62.861 rows=333333.33 loops=3)		
10	Buffers: shared hit=8335		
11	-> Parallel Seq Scan on porudzbina_p0 porudzbina_1 (cost=0.00..3566.90 rows=147590 width=6) (actual time=0.008..34.178 rows=250903.00 loo...		
12	Buffers: shared hit=2091		
13	-> Parallel Seq Scan on porudzbina_p1 porudzbina_2 (cost=0.00..3558.16 rows=147216 width=6) (actual time=0.012..33.060 rows=250267.00 loo...		
14	Buffers: shared hit=2086		
15	-> Parallel Seq Scan on porudzbina_p2 porudzbina_3 (cost=0.00..3552.97 rows=146997 width=6) (actual time=0.008..10.569 rows=83298.33 loo...		
16	Buffers: shared hit=2083		
17	-> Parallel Seq Scan on porudzbina_p3 porudzbina_4 (cost=0.00..3539.32 rows=146432 width=6) (actual time=0.011..29.474 rows=248935.00 loo...		
18	Buffers: shared hit=2075		
19	Planning:		
20	Buffers: shared hit=9		
21	Planning Time: 0.273 ms		Acti
22	Execution Time: 139.818 ms		Go to

Rezultati merjenja sumirani su u nastavku:

Upit	porudzbina_obicna (bez particija)	porudzbina (4 heš-particije)
Ciljani upit (kupac_id = 42): count + sum	104,518 ms	38,408 ms (eliminacija particija — skenirana samo p2)
Netargetirani upit: count(*) WHERE status = 'NOVA'	144,505 ms	236,885 ms (Parallel Append preko svih p0–p3)
Globalna agregacija: sum(iznos) nad celom tabelom	129,832 ms	139,818 ms (Parallel Append preko svih p0–p3)

Tabela 4. Rezultati merjenja: particionisana naspram neparticionisane tabelle (1.000.000 redova)

Ovaj naizgled kontraintuitivan rezultat direktno potvrđuje teorijsko razmatranje iz odeljka 5.1: particionisanj ubrzava isključivo ciljane upite koji mogu iskoristiti eliminaciju particija. Kada uslov upita ne sadrži ključ particionisanja, sistem mora da otvori i obradi svaku particiju pojedinačno, a zatim objedini parcijalne rezultate — u ovom slučaju kroz lokalni ekvivalent scatter-gather obrasca, sproveden unutar jedne instance — što unosi dodatni režijski trošak (planiranje i koordinaciju paralelnih radnika, agregaciju delimičnih rezultata) bez ijedne prednosti u odnosu na prosto sekvencijalno skeniranje jedne, nepodeljene tabele.

Particionisanje, drugim rečima, nije opšte ubrzanje čitanja, već ciljana optimizacija čija korist zavisi isključivo od toga da li upiti u praksi sadrže kolonu particionisanja u uslovu. Ista pouka ponoviće se, u znatno izraženijem obliku, i u odeljku 7.2, pri poređenju ciljanog i scatter-gather upita preko mreže — gde je cena netargetiranog upita još veća, jer se parcijalni rezultati moraju prenositi između fizički odvojenih servera, a ne samo između paralelnih radnika iste instance.

7.2. Sharding preko više servera pomoću `postgres_fdw`

Deklarativno particionisanje postaje pravi sharding kada se particije fizički izmeste na druge servere. PostgreSQL to omogućava kombinovanjem particionisanja sa mehanizmom **stranih tabela** (engl. foreign data wrapper): particija roditeljske tabele može biti strana tabela koja upućuje na tabelu u drugoj, udaljenoj PostgreSQL instanci [12]. Time koordinatorska instanca postaje ruter, a podaci žive na udaljenim čvorovima.

Pretpostavimo da pored lokalne (koordinatorske) instance postoje dva radna servera, `shard1` i `shard2`. Na svakom radnom serveru kreira se fizička tabela identične strukture:

```
-- izvršiti na serverima shard1 i shard2
CREATE TABLE porudzbina_lokal (
    id          bigint NOT NULL,
    kupac_id    bigint NOT NULL,
    datum       timestamptz NOT NULL,
    iznos       numeric(12,2) NOT NULL,
    status      text NOT NULL,
    PRIMARY KEY (id, kupac_id)
);
```

Na koordinatoru se zatim aktivira ekstenzija, registruju udaljeni serveri i korisnička preslikavanja, i kreira particionisana tabela čije su particije strane tabele:

```
CREATE EXTENSION postgres_fdw;

CREATE SERVER shard1 FOREIGN DATA WRAPPER postgres_fdw
    OPTIONS (host '10.0.0.11', port '5432', dbname 'prodaja');
```

```

CREATE SERVER shard2 FOREIGN DATA WRAPPER postgres_fdw
  OPTIONS (host '10.0.0.12', port '5432', dbname 'prodaja');

CREATE USER MAPPING FOR app_user SERVER shard1
  OPTIONS (user 'app_user', password '*****');
CREATE USER MAPPING FOR app_user SERVER shard2
  OPTIONS (user 'app_user', password '*****');

CREATE TABLE porudzbina (
  id          bigint NOT NULL,
  kupac_id    bigint NOT NULL,
  datum       timestampz NOT NULL,
  iznos       numeric(12,2) NOT NULL,
  status      text NOT NULL
) PARTITION BY HASH (kupac_id);

CREATE FOREIGN TABLE porudzbina_s1 PARTITION OF porudzbina
  FOR VALUES WITH (MODULUS 2, REMAINDER 0)
  SERVER shard1 OPTIONS (table_name 'porudzbina_lokal');

CREATE FOREIGN TABLE porudzbina_s2 PARTITION OF porudzbina
  FOR VALUES WITH (MODULUS 2, REMAINDER 1)
  SERVER shard2 OPTIONS (table_name 'porudzbina_lokal');

```

The screenshot shows a PostgreSQL client window titled 'prodaja/postgres@coordinate'. The interface includes a toolbar with icons for file operations, query execution, and settings. Below the toolbar, there are tabs for 'Query', 'Query History', and 'AI Assistant'. The 'Query' tab is active, displaying a list of SQL queries numbered 1 through 13. The queries are as follows:

```

1 CREATE EXTENSION postgres_fdw;
2
3 CREATE SERVER shard1 FOREIGN DATA WRAPPER postgres_fdw
4   OPTIONS (host 'shard1', port '5432', dbname 'prodaja');
5
6 CREATE SERVER shard2 FOREIGN DATA WRAPPER postgres_fdw
7   OPTIONS (host 'shard2', port '5432', dbname 'prodaja');
8
9 CREATE USER MAPPING FOR postgres SERVER shard1
10  OPTIONS (user 'postgres', password 'lozinka');
11
12 CREATE USER MAPPING FOR postgres SERVER shard2
13  OPTIONS (user 'postgres', password 'lozinka');

```

Below the queries, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is active, showing the following output:

```

CREATE USER MAPPING

```

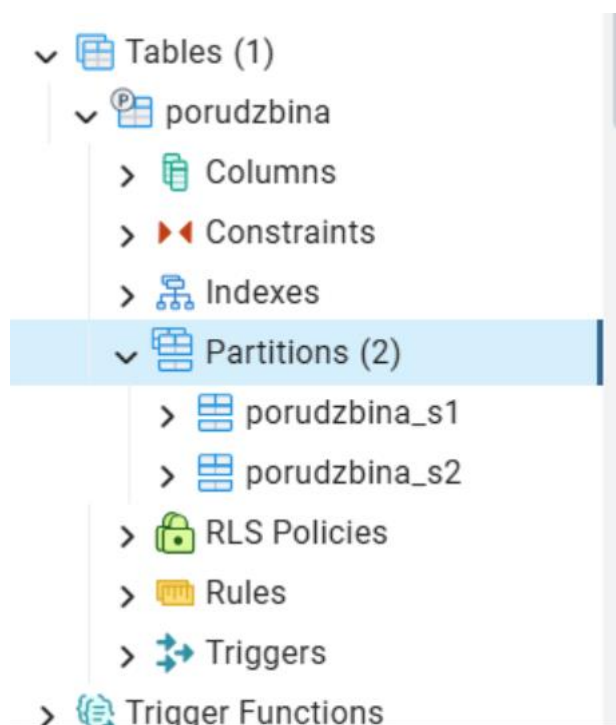
At the bottom, a status message indicates: 'Query returned successfully in 153 msec.'

Aplikacija sada radi isključivo sa logičkom tabelom porudzbina na koordinatoru: INSERT se automatski usmerava na odgovarajući udaljeni server na osnovu heš vrednosti kupac_id, a SELECT sa uslovom po ključu raspodele izvršava se samo na jednom šardu. PostgreSQL planer ume da spusti (engl. push down) uslove filtriranja, sortiranja, spajanja i agregacije na udaljene servere, što se u EXPLAIN planu vidi kao Foreign Scan sa udaljenim SQL upitom.

Ovaj pristup je potpuno standardan, bez spoljnih zavisnosti, i sasvim upotrebljiv za umerene scenarije, ali ima jasna ograničenja koja odgovaraju teorijskim izazovima iz poglavlja 5: nema automatskog

rebalansiranja (promena broja šardova zahteva ručno premeštanje podataka), koordinador je usko grlo i jedinstvena tačka otkaza, a transakcije preko više stranih servera nemaju punu 2PC garanciju u standardnoj konfiguraciji.

U pgAdmin-u se sve navedene komponente vide u stablu objekata: Foreign Data Wrappers → postgres_fdw → Foreign Servers (shard1, shard2) sa pripadajućim User Mappings, dok se strane tabele prikazuju kao particije roditeljske tabele. Registrovanjem samih radnih servera kao zasebnih konekcija u pgAdmin-u može se direktno proveriti fizički sadržaj svakog šarda.



Particionisana tabela je zatim napunjena sa 1.000.000 sintetičkih redova, pri čemu je INSERT izvršen preko koordinatora:

```
Query  Query History  AI Assistant
1  INSERT INTO porudzbina (id, kupac_id, datum, iznos, status)
2  SELECT g,
3      (random() * 100000)::bigint + 1,
4      now() - (random() * interval '365 days'),
5      round((random() * 5000)::numeric, 2),
6      'NOVA'
7  FROM generate_series(1, 1000000) AS g;
```

Data Output Messages Notifications

INSERT 0 1000000

Query returned successfully in 2 min 15 secs.

Umetanje je trajalo 2 minuta i 15 sekundi — znatno duže nego ekvivalentno umetanje u lokalno particionisanu tabelu iz odeljka 7.1, jer svaki red mora mrežnim putem da se prenese sa koordinatora na odgovarajući udaljeni server pre nego što se fizički upiše.

Da bi se proverila stvarna raspodela podataka, broj redova je izbrojan direktno na svakom radnom serveru, kroz zasebne pgAdmin konekcije na shard1 i shard2:

```
Query  Query History  AI Assistant
1  SELECT count(*) FROM porudzbina_lokal;
```

Data Output Messages Notifications

count bigint

1	499971
---	--------

The screenshot shows a PostgreSQL query client interface. At the top, the connection is labeled 'prodaja/postgres@shard2'. Below this is a toolbar with icons for file operations, a filter icon, and a 'No limit' dropdown. The main area is divided into tabs: 'Query', 'Query History', and 'AI Assistant'. The 'Query' tab is active, showing a single SQL query: `1 SELECT count(*) FROM porudzbina_lokal;`. Below the query editor, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, displaying a table with the query results. The table has two columns: 'count' (type 'bigint') and a value '500029'.

	count bigint
1	500029

Na serveru shard1 upit je vratio 499.971 red, a na serveru shard2 500.029 redova. Raspodela je gotovo savršeno ravnomerna (razlika od svega 58 redova, odnosno 0,006% od ukupnog broja), što potvrđuje kvalitet heš funkcije po kojoj je particionisanje izvedeno (MODULUS 2) — svaki od dva servera nezavisno čuva blizu polovine ukupnog skupa podataka, u skladu sa teorijskim očekivanjem iz odeljka 4.2.

Konačno, ciljani upit sa uslovom po ključu raspodele proveren je komandom EXPLAIN, izvršenom na koordinatorskom serveru:

The screenshot shows a PostgreSQL client interface with the connection 'prodaja/postgres@coordinate'. The query editor contains the following SQL:

```
1 EXPLAIN (ANALYZE, COSTS OFF)
2 SELECT * FROM porudzbina WHERE kupac_id = 42;
```

The 'Data Output' tab is active, displaying the 'QUERY PLAN' for the executed query. The plan shows a 'Foreign Scan on porudzbina_s1 porudzbina' with an actual time of 25.486..25.488 ms and 12 rows. The planning time was 0.906 ms and the execution time was 26.131 ms.

	QUERY PLAN
1	Foreign Scan on porudzbina_s1 porudzbina (actual time=25.486..25.488 rows=12 loop...
2	Planning Time: 0.906 ms
3	Execution Time: 26.131 ms

Plan pokazuje Foreign Scan on porudzbina_s1 porudzbina, sa vremenom izvršavanja od 26,131 ms. Planer je ispravno prepoznao da vrednost kupac_id = 42 pripada opsegu particije porudzbina_s1 (shard1), pa je upit u potpunosti prosleđen (engl. pushdown) tom serveru, bez ikakvog pristupa serveru shard2 — lokalni ekvivalent eliminacije particija iz odeljka 7.1, sada primenjen preko mreže.

Ovi rezultati zajedno potvrđuju da postgres_fdw predstavlja funkcionalan, iako ne i performansno besplatan, mehanizam za sharding: raspodela podataka je ravnomerna (499.971 : 500.029), a ciljani upiti se ispravno usmeravaju na tačan server bez nepotrebnog skeniranja ostatka klastera. Cena ovog pristupa vidljiva je u dva mesta — u produženom vremenu umetanja (2 min 15 sek za milion redova, naspram nekoliko sekundi u lokalnom slučaju) i u latenciji ciljanog upita (26,1 ms naspram manje od 1 ms lokalno), pri čemu oba efekta potiču isključivo od mrežne komunikacije koordinatora sa udaljenim serverima, a ne od same particione logike, koja radi identično kao i u okviru jedne instance.

Radi poređenja, izvršen je i upit koji ne sadrži uslov po ključu raspodele — globalna agregacija nad celom tabelom:

The screenshot shows a PostgreSQL query interface. At the top, the connection is 'prodaja/postgres@coordinate'. Below the connection bar, there are tabs for 'Query', 'Query History', and 'AI Assistant'. The 'Query' tab is active, showing a SQL query:

```
1 EXPLAIN (ANALYZE, COSTS OFF)
2 SELECT count(*), sum(iznos) FROM porudzbina;
```

Below the query, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing the 'QUERY PLAN' for the query. The plan is as follows:

Step	Operation
1	Aggregate (actual time=1981.483..1981.485 rows=1 loops=1)
2	-> Append (actual time=0.650..1891.032 rows=1000000 loops=1)
3	-> Foreign Scan on porudzbina_s1 porudzbina_1 (actual time=0.648..962.811 rows=499971 loop...)
4	-> Foreign Scan on porudzbina_s2 porudzbina_2 (actual time=0.471..877.685 rows=500029 loop...)
5	Planning Time: 0.795 ms
6	Execution Time: 1982.530 ms

Za razliku od prethodnog ciljanog upita, plan ovde pokazuje čvor Append koji objedinjuje rezultate sa oba udaljena servera — Foreign Scan on porudzbina_s1 (499.971 vraćenih redova) i Foreign Scan on porudzbina_s2 (500.029 redova) — sa ukupnim vremenom izvršavanja od 1.982,53 ms. Ovo je praktična ilustracija scatter-gather obrasca opisanog u odeljku 5.1: pošto upit ne sadrži ključ raspodele, koordinador nema način da unapred zna na kom se serveru nalaze traženi podaci, pa mora da pošalje upit na sve šardove i sam objedini parcijalne rezultate (u ovom slučaju, sabere delimične sume i brojeve redova sa oba servera). Vreme izvršavanja ovog upita ($\approx 1,98$ s) je više od 75 puta duže od ciljanog upita (26,1 ms), što jasno pokazuje cenu koju plaćaju upiti bez ključa raspodele u shardovanom sistemu — direktna potvrda preporuke iz odeljka 3.1 da najčešći upiti aplikacije treba da sadrže ključ raspodele u uslovu.

7.3. Citus: distribuirani PostgreSQL klaster

Za produkcione scenarije sa zahtevima za automatskim upravljanjem šardovima, paralelnim upitima i rebalansiranjem, standardno rešenje u PostgreSQL ekosistemu je ekstenzija Citus. Za potrebe eksperimenta klaster je podignut alatom Docker Compose, sa tri servisa koji predstavljaju koordinador (master) i dva radna čvora (worker1, worker2), povezana u zajedničku Docker mrežu:

```
services:
  master:
```

```
image: citusdata/citus:13.0
container_name: citus_master
hostname: master
ports:
  - "5500:5432"
environment:
  POSTGRES_PASSWORD: lozinka
  POSTGRES_DB: prodaja
  POSTGRES_HOST_AUTH_METHOD: trust
networks:
  - citusnet
```

```
worker1:
  image: citusdata/citus:13.0
  container_name: citus_worker1
  hostname: worker1
  environment:
    POSTGRES_PASSWORD: lozinka
    POSTGRES_DB: prodaja
    POSTGRES_HOST_AUTH_METHOD: trust
  networks:
    - citusnet
```

```
worker2:
  image: citusdata/citus:13.0
  container_name: citus_worker2
  hostname: worker2
  environment:
    POSTGRES_PASSWORD: lozinka
    POSTGRES_DB: prodaja
    POSTGRES_HOST_AUTH_METHOD: trust
  networks:
    - citusnet
```

```
networks:
  citusnet:
    driver: bridge
```

Nakon pokretanja kontejnera (docker compose up -d), na koordinatorskom čvoru je aktivirana ekstenzija i registrovani su radni čvorovi:

```
CREATE EXTENSION IF NOT EXISTS citus;  
  
SELECT citus_add_node('worker1', 5432);  
SELECT citus_add_node('worker2', 5432);
```

The screenshot shows a SQL query editor with three tabs: "Query", "Query History", and "AI Assistant". The "Query" tab is active, displaying a SQL script with line numbers 1 through 6. The script consists of three commands: creating the Citus extension, adding two worker nodes, and querying the active worker nodes. Below the query editor, there are tabs for "Data Output", "Messages", and "Notifications". The "Data Output" tab is active, showing a table of results. The table has two columns: "node_name" (text) and "node_port" (bigint). The results show two worker nodes: "worker2" at port 5432 and "worker1" at port 5432.

	node_name text	node_port bigint
1	worker2	5432
2	worker1	5432

Zatim je kreirana radna šema sa tabelama kupaca i porudžbina, pri čemu je tabela porudzbina distribuirana po istom ključu kao i kupac (kolokacija):

prodaja/postgres@Citrus Klaster

Query Query History AI Assistant

```

1 CREATE TABLE kupac (
2     id         bigint PRIMARY KEY,
3     ime        text NOT NULL,
4     grad       text
5 );
6
7 CREATE TABLE porudzbina (
8     id          bigint NOT NULL,
9     kupac_id    bigint NOT NULL REFERENCES kupac(id),
10    datum       timestampz NOT NULL DEFAULT now(),
11    iznos        numeric(12,2) NOT NULL,
12    PRIMARY KEY (id, kupac_id)
13 );
14
15 SELECT create_distributed_table('kupac', 'id');
16 SELECT create_distributed_table('porudzbina', 'kupac_id',
17     colocate_with => 'kupac');

```

Data Output Messages Notifications

Showing row:

	create_distributed_table
1	void

Citus je podrazumevano kreirao 32 šarda po tabeli, raspoređena po radnim čvorovima. Tabela je zatim napunjena sa 100.000 kupaca i 10.000.000 porudžbina:

prodaja/postgres@Citrus Klaster

Query Query History AI Assistant

```

1 -- kupci (100.000)
2 INSERT INTO kupac (id, ime, grad)
3 SELECT g, 'Kupac ' || g,
4     (ARRAY['Niš', 'Beograd', 'Novi Sad', 'Kragujevac', 'Subotica'])[1 + (g % 5)]
5 FROM generate_series(1, 100000) AS g;
6
7 -- porudzbine (10.000.000)
8 INSERT INTO porudzbina (id, kupac_id, datum, iznos)
9 SELECT g,
10     (random() * 99999)::bigint + 1,
11     now() - (random() * interval '365 days'),
12     round((random() * 5000)::numeric, 2)
13 FROM generate_series(1, 10000000) AS g;

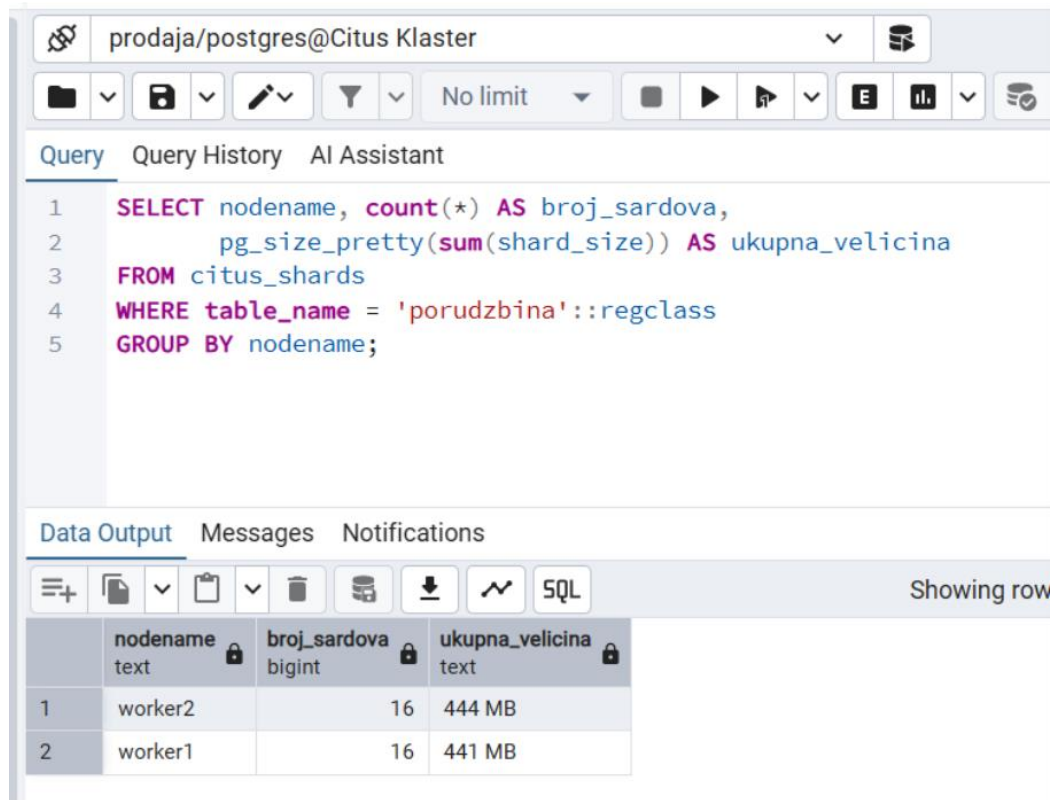
```

Data Output Messages Notifications

INSERT 0 10000000

Query returned successfully in 1 min 28 secs.

Raspodela nastalih šardova proverena je upitom nad Citus katalogom:



The screenshot shows a database query interface with the following components:

- Header:** Connection name 'prodaja/postgres@Citus Klaster'.
- Toolbar:** Includes icons for file operations, filters, and execution controls. A dropdown menu is set to 'No limit'.
- Query Editor:** Contains the following SQL query:

```
1 SELECT nodename, count(*) AS broj_sardova,  
2     pg_size_pretty(sum(shard_size)) AS ukupna_velicina  
3 FROM citus_shards  
4 WHERE table_name = 'porudzbina'::regclass  
5 GROUP BY nodename;
```
- Data Output:** A table with 4 columns: 'nodename' (text), 'broj_sardova' (bigint), and 'ukupna_velicina' (text). The table shows 2 rows of data.

	nodename text	broj_sardova bigint	ukupna_velicina text
1	worker2	16	444 MB
2	worker1	16	441 MB

Rezultat potvrđuje ravnomernu raspodelu: po 16 šardova na svakom čvoru, ukupne veličine 441 MB (worker1) i 444 MB (worker2) - praktična potvrda teorijskog očekivanja iz odeljka 4.2, da kvalitetna heš funkcija raspoređuje podatke bez izraženih vrućih tačaka.

Efekat kolokacije proveren je spajanjem tabela kupac i porudzbina po kupac_id, za jednog konkretnog kupca:

Query Query History AI Assistant

```

1 EXPLAIN (ANALYZE, COSTS OFF)
2 SELECT k.grad, count(*), sum(p.iznos)
3 FROM kupac k
4 JOIN porudzbina p ON p.kupac_id = k.id
5 WHERE k.id = 42

```

Data Output Messages Notifications

Showing rows: 1 to 25

	QUERY PLAN text
10	-> Sort (actual time=14.138..17.037 rows=115 loops=1)
11	Sort Key: k.grad
12	Sort Method: quicksort Memory: 28kB
13	-> Nested Loop (actual time=0.838..16.972 rows=115 loops=1)
14	-> Index Scan using kupac_pkey_102035 on kupac_102035 k (actual time=0.041..0.051 rows=1 loo...
15	Index Cond: (id = 42)
16	-> Gather (actual time=0.794..16.897 rows=115 loops=1)
17	Workers Planned: 1
18	Workers Launched: 1
19	-> Parallel Seq Scan on porudzbina_102067 p (actual time=0.229..10.773 rows=58 loops=2)
20	Filter: (kupac_id = 42)
21	Rows Removed by Filter: 155360
22	Planning Time: 0.509 ms
23	Execution Time: 18.100 ms
24	Planning Time: 1.137 ms
25	Execution Time: 21.350 ms

Total rows: 25 Query complete 00:00:00.083

Plan izvršavanja pokazuje da je upit pristupio samo jednom paru šardova (kupac_102035 i porudzbina_102067), koji se spajaju lokalno (Nested Loop sa Index Scan i Parallel Seq Scan), bez ikakvog premeštanja međurezultata preko mreže (engl. data shuffling). Ukupno vreme izvršavanja iznosilo je 21,35 ms. Ovo je direktna potvrda koncepta kolokacije opisanog u odeljku 3.3: pošto su obe tabele shardovane po istom ključu (id odnosno kupac_id), svi podaci jednog kupca fizički se nalaze na istom čvoru, pa se spajanje izvršava bez mrežne komunikacije.

Dodatno je kreirana i referentna (šifarnička) tabela statusa porudžbine, koja se u potpunosti replicira na sve radne čvorove:

QueryQuery HistoryAI Assistant

```
1 CREATE TABLE status_sifra (  
2     kod      text PRIMARY KEY,  
3     opis     text NOT NULL  
4 );  
5 INSERT INTO status_sifra VALUES  
6     ('NOVA', 'Nova porudžbina'),  
7     ('POSLATA', 'Poslata kupcu'),  
8     ('ZAVRSENA', 'Realizovana porudžbina');  
9  
10 SELECT create_reference_table('status_sifra');  
11  
12 SELECT nodename, count(*)  
13 FROM citus_shards  
14 WHERE table_name = 'status_sifra'::regclass  
15 GROUP BY nodename;
```

Data OutputMessagesNotifications

+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

St

	nodename text	count bigint
1	worker1	1
2	worker2	1

Provera potvrđuje da svaki od dva radna čvora poseduje po jedan (kompletan) šard tabele `status_sifra`, čime se omogućava lokalno spajanje sa bilo kojim šardom porudžbina, bez mrežnog premeštanja podataka.

Za razliku od particionisanja unutar jedne instance (odeljak 7.1), gde sve particije dele isti disk i isti procesor pa netargetirani upiti ne dobijaju nikakvu prednost, u Citus klasteru se šardovi fizički nalaze na odvojenim čvorovima sa sopstvenim procesorom, memorijom i diskom. Zbog toga se upiti bez ključa raspodele — na primer, globalna agregacija nad celom tabelom porudzbina — ovde stvarno paralelizuju preko nezavisnih resursa: koordinator šalje delimični upit na `worker1` i `worker2` istovremeno, a svaki od njih ga izvršava nad svojom polovinom podataka koristeći sopstveni CPU i I/O. To je suštinska razlika u odnosu na lokalno particionisanje: dok particije na jednoj instanci samo dodaju režijski trošak bez ikakve koristi za netargetirane upite (odeljak 7.1), sharding preko više servera pretvara isti scatter-gather obrazac u priliku za pravi paralelizam, jer se posao zaista deli između nezavisnih mašina.

7.4. Rad u alatu pgAdmin — rezime

Kroz prethodne odeljke pgAdmin je korišćen kao primarno radno okruženje. Za rad sa horizontalno raspoređenim podacima najkorisnije su sledeće mogućnosti alata: registrovanje više server-konekcija (koordinator i svaki radni čvor zasebno), čime se može uporediti logički pogled na podatke sa fizičkim sadržajem šardova; kartica Partitions u dijalogu za kreiranje tabela; prikaz stranih servera i stranih tabela u Object Explorer-u; grafički Explain prikaz u Query Tool-u, na kome se jasno vidi eliminacija particija odnosno Foreign Scan čvorovi; i ugrađeni Dashboard za praćenje aktivnih sesija po serveru, koristan pri posmatranju raspodele opterećenja tokom eksperimenata.

8. Prednosti, nedostaci i preporuke za primenu

8.1. Prednosti

- **Skalabilnost kapaciteta i propusnosti** — obim podataka i broj operacija (naročito pisanja) mogu rasti približno linearno sa brojem čvorova, daleko iznad granica jedne mašine.
- **Bolje performanse ciljanih upita** — svaki šard održava manje tabele i manje indekse, pa su lokalne operacije brže; opterećenje se raspodeljuje na više procesora, memorija i diskova.
- **Ograničavanje dometa otkaza** — otkaz jednog čvora (uz replikaciju šarda) pogađa samo deo podataka, umesto celog sistema.
- **Ekonomičnost** — klaster standardnih servera je po pravilu jeftiniji od ekvivalentno moćnog pojedinačnog servera visoke klase.
- **Geografska fleksibilnost** — podaci se mogu smestiti bliže korisnicima, uz poštovanje regulative o lokalizaciji.

8.2. Nedostaci

- **Složenost projektovanja** — izbor ključa raspodele je kritična i teško reverzibilna odluka; loš izbor vodi vrućim tačkama i skupim scatter-gather upitima.
- **Skupe među-šard operacije** — spajanja, globalne agregacije, globalna ograničenja jedinstvenosti i distribuirane transakcije zahtevaju mrežnu koordinaciju i posebne tehnike (kolokacija, denormalizacija, 2PC).
- **Operativni teret** — rezervne kopije, migracije šeme, nadzor i otklanjanje problema postaju distribuirani problemi; rebalansiranje zahteva pažljivo planiranje.
- **Ograničenja alata i ekosistema** — deo standardnih mogućnosti DBMS-a (sekvence, triggeri, određena ograničenja integriteta) u distribuiranom režimu radi ograničeno ili drugačije.

8.3. Kada (ne) uvoditi sharding

Iskustvena preporuka, dosledno prisutna i u literaturi i u inženjerskoj praksi, glasi da je sharding poslednja, a ne prva mera skaliranja [1, 2]. Pre njegovog uvođenja treba iscrpeti jednostavnije opcije, redom: optimizaciju upita i indeksa, vertikalno skaliranje, keširanje na aplikativnom nivou, replike za rasterećenje čitanja i particionisanje unutar jedne instance. Sharding postaje opravdan kada obim podataka ili propusnost pisanja trajno prevazilaze mogućnosti jednog servera, ili kada zahtevi geografske distribucije to nameću. Ako se uvodi, ključne odluke — ključ raspodele, strategija, kolokacija — treba donositi na osnovu stvarnih obrazaca pristupa podacima, uz projektovanje šeme tako da dominantna većina upita i transakcija ostane unutar jednog šarda.

9. Zaključak

Horizontalno pozicioniranje podataka predstavlja fundamentalnu tehniku skaliranja savremenih sistema za upravljanje bazama podataka. Deljenjem logičke baze na šardove raspoređene preko više nezavisnih servera prevazilaze se fizičke granice pojedinačne mašine i omogućava približno linearan rast kapaciteta i propusnosti — što je preduslov funkcionisanja praktično svih velikih internet sistema današnjice.

Analiza sprovedena u radu pokazuje da uspeh shardovanog sistema presudno zavisi od jedne rane projektne odluke: izbora ključa raspodele. Ključ visoke kardinalnosti, ravnomerne raspodele i usklađen sa dominantnim obrascima pristupa omogućava da se većina upita i transakcija izvršava lokalno, na jednom šardu; svako odstupanje od tog ideala plaća se scatter-gather upitima, mrežnim premeštanjem podataka i distribuiranim transakcijama. Strategije raspodele — opsegovna, heš, konzistentno heširanje, imenička — nude različite kompromise između ravnomernosti, efikasnosti opsegovnih upita i cene rebalansiranja, a savremeni sistemi ih često kombinuju.

Pregled savremenih DBMS rešenja pokazuje jasnu evolutivnu putanju: od ručno shardovanih relacionih baza, preko NoSQL sistema koji su sharding ugradili u srž arhitekture žrtvujući delove relacionog modela, do NewSQL sistema koji horizontalnu raspodelu čine potpuno transparentnom uz očuvanje SQL-a i ACID garancija. Praktični deo rada pokazao je da se i klasičan relacioni sistem poput PostgreSQL-a, kombinovanjem deklarativnog particionisanja, mehanizma stranih tabela i ekstenzije Citus, može postepeno transformisati od jednoserverske baze do distribuiranog klastera sa automatskim rebalansiranjem — pri čemu svaki od tih koraka verno reprodukuje teorijske koncepte i izazove opisane u radu.

Može se zaključiti da sharding nije univerzalno rešenje već inženjerski kompromis: značajan dobitak

u skalabilnosti plaća se složenošću projektovanja i eksploatacije. Njegova primena je opravdana tek kada su jednostavnije tehnike iscrpljene, a tada zahteva pažljivo, na stvarnim obrascima pristupa zasnovano projektovanje. Trend razvoja NewSQL i distribuiranih SQL sistema, međutim, nagoveštava budućnost u kojoj će ta složenost u sve većoj meri biti skrivena unutar samog DBMS-a, a horizontalna raspodela postati podrazumevano, transparentno svojstvo baza podataka.

Literatura

- [1] M. Kleppmann, Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems, O'Reilly Media, 2017.
- [2] M. T. Özsu, P. Valduriez, Principles of Distributed Database Systems, 4. izdanje, Springer, 2020.
- [3] P. Bailis et al., Readings in Database Systems (Red Book), 5. izdanje, MIT, 2015.
- [4] Citus Data / Microsoft, Citus Documentation — Distributed PostgreSQL, <https://docs.citusdata.com> (pristupljeno: jun 2026).
- [5] J. C. Corbett et al., „Spanner: Google's Globally-Distributed Database”, Proceedings of OSDI 2012, str. 251–264, 2012.
- [6] D. Karger et al., „Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web”, Proceedings of STOC 1997, str. 654–663, 1997.
- [7] A. Lakshman, P. Malik, „Cassandra: A Decentralized Structured Storage System”, ACM SIGOPS Operating Systems Review, vol. 44, br. 2, str. 35–40, 2010.
- [8] J. Gray, A. Reuter, Transaction Processing: Concepts and Techniques, Morgan Kaufmann, 1992.
- [9] S. Gilbert, N. Lynch, „Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services”, ACM SIGACT News, vol. 33, br. 2, str. 51–59, 2002.
- [10] MongoDB Inc., MongoDB Manual — Sharding, <https://www.mongodb.com/docs/manual/sharding/> (pristupljeno: jun 2026).
- [11] The Vitess Authors, Vitess Documentation, <https://vitess.io/docs/> (pristupljeno: jun 2026).
- [12] The PostgreSQL Global Development Group, PostgreSQL 17 Documentation — Table Partitioning; postgres_fdw, <https://www.postgresql.org/docs/> (pristupljeno: jun 2026).